Continuous time Trajectory Estimation for Quadcopters.

Shiladitya Biswas* and Jun Chen†
San Diego State University, San Diego, CA 92182

Vision-based state estimation for unmanned aerial vehicles (UAVs) like quadcopter must provide accurate and globally consistent trajectories within the limited computational resources available onboard. Recent learning-based visual geometry models, such as VGGT-Ω, infer camera poses directly from images, but they provide estimates only at discrete image times, encode no knowledge of vehicle dynamics, and require substantial graphics processing unit (GPU) memory. This paper proposes a continuous-time trajectory estimation framework for monocular quadcopter navigation based on Gaussian-process (GP) regression. The trajectory is modeled with a white-noise-on-acceleration motion prior on a uniform temporal grid, and observations from all image frames are incorporated through GP interpolation, so that the pose and velocity can be queried at any time while the estimation problem remains sparse. The framework is evaluated on three sequences of the European Robotics Challenge (EuRoC) micro aerial vehicle dataset against ORB-SLAM3, a keyframe-based GP variant, and VGGT-Ω under a common evaluation protocol. Preliminary results show that the proposed framework reduces the absolute trajectory error of its ORB-SLAM3 initialization by 24% to 31% while matching its rotation accuracy, and achieves an error 22 to 50 times lower than that of VGGT-Ω without requiring a GPU. The results also show that placing the GP states on a uniform grid, rather than at front-end keyframes, is essential to avoid large interpolation errors.

I. Introduction

Accurate and reliable state estimation is a fundamental requirement for the autonomous operation of unmanned aerial vehicles (UAVs). Quadcopter UAVs are increasingly deployed for inspection, mapping, search and rescue, and delivery, often in environments where Global Navigation Satellite System (GNSS) signals are degraded or unavailable. In such settings, onboard cameras offer a lightweight, low-power, and information-rich sensing modality, and monocular visual odometry and simultaneous localization and mapping (SLAM) have become central components of vision-based UAV navigation [1–3]. An estimator intended for this purpose must provide an accurate and globally consistent trajectory over a complete flight, and it must do so within the limited computational and memory resources of a small aerial platform.

Most visual estimators adopt a discrete-time formulation, in which the vehicle state is estimated only at the times at which images are acquired or at a subset of keyframes [2, 4]. This formulation does not model the vehicle motion between those times, although the state between images is needed for control, for fusion with asynchronous sensors, and for evaluating the trajectory at arbitrary times. Recently, learning-based visual geometry models have emerged as an alternative to such model-based pipelines. Feed-forward networks such as DUSt3R [5], VGGT [6], and VGGT-Ω [7] predict camera poses and dense scene geometry directly from images, without hand-designed features, explicit motion models, or, in many cases, camera calibration. However, these models also estimate poses only at discrete image times, their priors are learned from general-purpose imagery rather than derived from vehicle dynamics, and their large transformer architectures require graphics processing units (GPUs) with substantial memory. Moreover, because long image sequences exceed the capacity of a single forward pass, the sequences must be processed in segments whose predictions are not guaranteed to be mutually consistent.

Continuous-time trajectory estimation addresses the first of these limitations by representing the trajectory as a function of time that can be evaluated at any instant [8]. In particular, modeling the trajectory as a Gaussian process (GP) allows the estimation problem to be posed as GP regression [9]. When the GP prior is generated by a linear time-varying stochastic differential equation, the resulting problem retains an exactly sparse structure, so it can be solved with a computational cost comparable to that of discrete-time methods, and the trajectory at any intermediate time can be obtained by interpolating between neighboring estimation times [10, 11]. The prior also encodes explicit

*Ph.D. Student, Department of Aerospace Engineering, sbiswas3597@sdsu.edu
†Associate Professor, Department of Aerospace Engineering, AIAA Senior Member, jun.chen@sdsu.edu

and physically interpretable assumptions on the smoothness of the vehicle motion [12], which regularize the estimate when measurements are noisy or sparse.

This paper proposes a GP regression-based continuous-time trajectory estimation framework for monocular quadcopter navigation. The vehicle trajectory is modeled with a white-noise-on-acceleration motion prior, and the poses and velocities are estimated jointly with the landmarks in a sparse factor graph that is initialized from a monocular ORB-SLAM3 front end [13]. Two design choices distinguish the framework. First, the GP states are placed on a uniform temporal grid rather than at the keyframes selected by the front end, which bounds the interval over which the motion prior must represent the vehicle motion. Second, observations from image frames between states are incorporated through GP interpolation, so that the entire image stream constrains the trajectory without adding a state variable for every image. It is hypothesized that the proposed framework will achieve higher accuracy than a state-of-the-art learned model at a substantially lower computational cost. This hypothesis is tested on three sequences of the European Robotics Challenge (EuRoC) micro aerial vehicle dataset [14], on which the framework is compared with ORB-SLAM3, with a variant that places the GP states at the keyframes, and with VGGT-Ω under a common evaluation protocol.

The main contributions of this work are as follows:

1) A GP regression-based continuous-time trajectory estimation framework for monocular quadcopter UAVs, in which states on a uniform temporal grid and interpolated reprojection factors provide pose and velocity estimates that can be queried at any time.
2) An experimental comparison of the proposed framework with ORB-SLAM3 and the learning-based VGGT-Ω model on the EuRoC dataset, in terms of absolute trajectory error, relative pose error, rotation error, and computational cost, using the same images, ground truth, and alignment for all methods.
3) An analysis of the effect of state placement, which shows that a uniform temporal grid avoids the large interpolation errors that arise when the GP states are placed at front-end keyframes.

The remainder of this paper is organized as follows. Section II reviews related work on continuous-time and learning-based trajectory estimation. Section III presents the proposed GP-based continuous-time estimation framework. Section IV presents the preliminary results, including the experimental setup, the comparison with ORB-SLAM3 and VGGT-Ω, and analyses of state placement and computational cost. Section V concludes the paper and outlines directions for future work.

II. Related Work

This section reviews the two families of trajectory estimation methods considered in this paper. Section II.A summarizes classical model-based visual estimation and its continuous-time extensions, with emphasis on Gaussian-process (GP) motion priors. Section II.B reviews learning-based approaches that infer camera motion and scene geometry directly from images, including the VGGT family of models. Section II.C positions the present work relative to both.

A. Model-Based and Continuous-Time Trajectory Estimation

Classical monocular visual odometry and SLAM systems estimate camera motion by minimizing geometric or photometric errors over a discrete set of keyframe poses. Feature-based methods such as ORB-SLAM [2], semi-direct methods such as SVO [3], and direct methods such as Direct Sparse Odometry (DSO) [15] have achieved high accuracy on aerial benchmarks, and visual-inertial extensions such as VINS-Mono [4] have become widely used on UAV platforms. In all of these systems, the trajectory is represented only at measurement times, and the vehicle motion between those times is not explicitly modeled.

Continuous-time formulations instead represent the trajectory as a function of time. One widely used approach parameterizes the trajectory with temporal basis functions, most commonly cumulative B-splines on Lie groups [8, 16, 17]. Spline-based methods naturally accommodate asynchronous and high-rate measurements, such as those from rolling-shutter and event cameras [18], but the knot spacing must be selected in advance and directly governs the trade-off between trajectory smoothness and the ability to represent rapid motion.

An alternative approach treats the trajectory as a GP and performs estimation as GP regression [9]. Barfoot et al. [10] showed that when the GP prior is generated by a linear time-varying stochastic differential equation, the inverse kernel matrix is exactly sparse, so batch estimation retains the computational efficiency of discrete-time methods while allowing the trajectory to be queried at any time through GP interpolation. This framework was extended to poses on SE(3) [11] and to matrix Lie groups more generally [19], and higher-order priors such as the white-noise-on-jerk model [12] have been proposed to better represent agile motion. A key advantage of GP priors is that the motion

model is physically interpretable: its hyperparameters correspond to the expected power spectral density of the vehicle acceleration or jerk. The corresponding limitation is that the prior is hand-designed, and its performance depends on how well these assumptions match the true vehicle dynamics and on the quality of the visual front end that supplies measurements.

B. Learning-Based Trajectory and Geometry Estimation

Learning-based methods replace some or all of the hand-designed components of the classical pipeline with networks trained on large datasets. Hybrid systems such as DROID-SLAM [20] combine learned dense correspondence with differentiable bundle adjustment and achieve substantially improved robustness relative to classical systems.

A more recent line of work removes explicit optimization from the core of the pipeline. DUSt3R [5] introduced end-to-end regression of dense 3D point maps from uncalibrated image pairs, from which camera poses and intrinsic parameters can be recovered, with a global alignment step used to fuse pairwise predictions over multiple views. MASt3R [21] extended this formulation with dense local feature matching. VGGT [6] generalized the approach to many views in a single forward pass: a transformer that alternates between frame-wise and global attention, combined with multiple task-specific heads, jointly predicts camera poses, depth maps, point maps, and 3D point tracks. VGGT-Ω [7] builds on this architecture by introducing register attention and a single dense prediction head, which allow the model to scale beyond 10 billion parameters while reducing memory consumption by approximately 70% and extending applicability to dynamic scenes.

Several methods target video sequences and dynamic scenes specifically. MonST3R [22] adapts point-map regression to scenes that contain moving objects, and MegaSaM [23] combines learned depth and optical-flow priors with differentiable bundle adjustment to estimate camera motion from casual, dynamic videos. A known limitation of single-pass feed-forward models is that their accuracy degrades over long sequences, in which scale and orientation drift accumulate. Methods such as HiP3R [24] and TrajVG [25] address this limitation through hierarchical pose graphs and trajectory-coupled geometry representations, respectively.

Compared with model-based estimators, these learned methods do not require explicit motion models, hand-tuned feature pipelines, or, in many cases, known camera calibration, and they can exploit geometric priors learned from large and diverse datasets. However, they produce pose estimates only at discrete image times, they are typically trained and evaluated on general-purpose reconstruction benchmarks rather than on aggressive aerial flight, and their computational and memory requirements can be substantial relative to the resources available onboard a small UAV.

Table 1 summarizes the learning-based methods reviewed in this section.

Table 1 Summary of learning-based visual geometry and trajectory estimation methods

Model or family	Core mechanism	Key strength
DUSt3R, MASt3R	Pairwise point-map regression with global alignment	End-to-end 3D point-map regression from uncalibrated image pairs
VGGT	Alternating frame-wise and global attention with multiple task heads	Single-pass prediction of camera poses, depth, point maps, and point tracks
VGGT-Ω	Register attention with a single dense prediction head	Scales beyond 10 billion parameters with reduced memory; handles dynamic scenes
MegaSaM, MonST3R	Learned depth and flow priors with differentiable optimization	Camera tracking in dynamic, non-rigid video
HiP3R, TrajVG	Hierarchical pose graphs and trajectory-coupled geometry	Reduced long-horizon scale and orientation drift

C. Positioning of This Work

Classical model-based systems such as ORB-SLAM [2], SVO [3], and DSO [15] are accurate and efficient enough to run on embedded processors, but they estimate the trajectory only at discrete times and therefore cannot directly provide the continuous-time pose and velocity estimates needed for control and asynchronous sensor fusion. Learning-based models such as VGGT [6] and VGGT-Ω [7] are general-purpose and can be applied off the shelf without hand-designed pipelines or, in many cases, camera calibration, but they also produce poses only at discrete image times, and their

large transformer architectures impose memory and computational requirements that make onboard deployment on small UAVs impractical at present. Continuous-time GP-based estimation combines the efficiency of classical methods, through an exactly sparse optimization problem [10, 11], with a trajectory that can be queried at any time and a motion prior that constrains the estimate to physically plausible vehicle motion [12]. The framework proposed in this paper builds on these properties to provide accurate, onboard-feasible continuous-time trajectory estimation for monocular quadcopter navigation, and it is evaluated against VGGT-Ω as a representative state-of-the-art learning-based baseline.

III. Methodology

This section presents the proposed continuous-time trajectory estimation framework. Section III.A gives an overview of the framework, and Section III.B briefly reviews Gaussian-process (GP) regression. Section III.C defines the trajectory states and their placement on a uniform temporal grid, Section III.D presents the GP motion prior and the corresponding interpolation scheme, and Section III.E describes the gauge and scale constraints. Section III.F introduces the landmark variables and the reprojection factors, and Section III.G summarizes the resulting optimization problem and its implementation.

A. Overview

The proposed framework formulates monocular trajectory estimation as a factor graph optimization problem [26] that combines a continuous-time GP motion prior with visual reprojection measurements. ORB-SLAM3 [13], run in monocular mode, is used as the visual front end. It provides an initial camera pose for every image frame, a set of three-dimensional (3D) landmarks, and the image observations of those landmarks in both keyframes and ordinary frames. Rather than placing estimation states at the keyframes selected by the front end, the trajectory is parameterized by states placed on a uniform temporal grid, and consecutive states are connected by a white-noise-on-acceleration (WNOA) motion prior. Each landmark observation constrains the trajectory either directly, when the observation coincides with a state time, or through GP interpolation between the two neighboring states otherwise. In this way, observations from all frames contribute to the estimate without introducing a state variable for every frame, and the optimized trajectory can be evaluated at any time. The factor graph is implemented and solved using the Georgia Tech Smoothing and Mapping (GTSAM) library [27].

B. Gaussian-Process Regression

A GP is a distribution over functions in which any finite set of function values is jointly Gaussian [28]. A GP over a continuous-time trajectory $\mathbf{x}(t)$ is written as

$$\mathbf{x}(t) \sim \mathcal{GP}\left(\check{\mathbf{x}}(t),\, \check{\mathbf{P}}(t, t')\right), \tag{1}$$

where $\check{\mathbf{x}}(t)$ is the prior mean function and $\check{\mathbf{P}}(t, t')$ is the prior covariance function, or kernel. Given measurements of the trajectory, GP regression computes the posterior distribution over the trajectory, which can then be evaluated at any time of interest. For general kernels, the covariance matrix over the estimation times is dense, and the cost of batch inference grows cubically with the number of states. However, when the GP prior is generated by a linear time-varying stochastic differential equation (SDE) driven by white noise, the prior is Markovian, and its inverse covariance matrix is exactly block-tridiagonal [10]. The resulting estimation problem retains the sparsity of discrete-time formulations, and the trajectory at any intermediate time can be obtained by interpolating between the two neighboring estimation times only. This property is extended to poses on the special Euclidean group SE(3) in [11], and it forms the basis of the framework presented here.

C. Trajectory State Parameterization

1. State Definition

The trajectory is parameterized by a set of discrete states,

$$\mathbf{x}_i = \{\mathbf{T}_i,\, \boldsymbol{\varpi}_i\}, \qquad i = 0, \ldots, N, \tag{2}$$

where $\mathbf{T}_i \in SE(3)$ is the pose of the camera frame with respect to the world frame at the state time t_i, and $\boldsymbol{\varpi}_i \in \mathbb{R}^6$ is the generalized velocity of the camera expressed in the camera frame, consisting of its angular and linear velocity

components. Because a monocular camera is the only sensor, the camera frame is used as the vehicle body frame, and no camera-to-body extrinsic transformation is required. All times are measured relative to the timestamp of the first image frame.

2. Uniform Temporal Grid

The states are placed on a uniform temporal grid with spacing Δt, independent of the keyframe selection of the front end. Since the camera operates at a fixed frame period δt, the grid is aligned with the image timestamps by assigning one state to every $s = \text{round}(\Delta t/\delta t)$-th frame, and an additional state is placed at the final frame so that every frame lies within a state interval. In this work, $\Delta t = 0.5$ s and $\delta t = 0.05$ s, so that one state is assigned to every tenth frame.

Placing the states on a uniform grid has two advantages over placing them at the front-end keyframes. First, the keyframe selection of ORB-SLAM3 is driven by tracking quality and scene content, which produces irregular intervals between keyframes, including long gaps during which the trajectory is represented only by the motion prior. A uniform grid instead provides a consistent temporal resolution along the entire trajectory and a consistent prior covariance $\mathbf{Q}(\Delta t)$ between all consecutive states. Second, it decouples the trajectory representation from the decisions of the front end, so that the spacing Δt becomes a single, interpretable design parameter that sets the temporal resolution of the trajectory, in combination with the motion prior.

3. Initialization

The state poses are initialized from the ORB-SLAM3 tracking poses of the frames at which the states are placed. The velocities are initialized by central finite differences between the ORB-SLAM3 poses of the two adjacent frames,

$$\boldsymbol{\varpi}_i \approx \frac{\text{Log}\left(\mathbf{T}_{j-1}^{-1}\,\mathbf{T}_{j+1}\right)}{t_{j+1} - t_{j-1}}, \tag{3}$$

where j is the frame index of state i, the frame poses $\mathbf{T}_{j\pm1}$ are taken from the front end, and $\text{Log}(\cdot)$ denotes the logarithmic map from SE(3) to $\mathbb{R}^6$ [29]. One-sided differences are used at the first and last frames.

D. White-Noise-on-Acceleration Motion Prior

Following [11], the trajectory between consecutive states is described using a local variable $\boldsymbol{\xi}_i(t) \in \mathbb{R}^6$ defined relative to the pose at time t_i,

$$\boldsymbol{\xi}_i(t) = \text{Log}\left(\mathbf{T}_i^{-1}\mathbf{T}(t)\right), \qquad t \in [t_i, t_{i+1}]. \tag{4}$$

The WNOA prior assumes that the second time derivative of this local variable is a zero-mean white-noise process,

$$\ddot{\boldsymbol{\xi}}_i(t) = \mathbf{w}(t), \qquad \mathbf{w}(t) \sim \mathcal{GP}\left(\mathbf{0},\, \mathbf{Q}_c\,\delta(t-t')\right), \tag{5}$$

where $\mathbf{Q}_c \in \mathbb{R}^{6\times6}$ is the power spectral density matrix of the process noise and $\delta(\cdot)$ is the Dirac delta function. The diagonal entries of $\mathbf{Q}_c$ are hyperparameters that encode the expected magnitude of the angular and linear accelerations of the vehicle, and they therefore provide a physically interpretable means of tuning the smoothness of the estimated trajectory. Defining the local state $\boldsymbol{\gamma}_i(t) = [\boldsymbol{\xi}_i(t)^T,\, \dot{\boldsymbol{\xi}}_i(t)^T]^T$, this prior corresponds to the linear time-invariant SDE

$$\dot{\boldsymbol{\gamma}}_i(t) = \begin{bmatrix}\mathbf{0} & \mathbf{1}\\ \mathbf{0} & \mathbf{0}\end{bmatrix}\boldsymbol{\gamma}_i(t) + \begin{bmatrix}\mathbf{0}\\ \mathbf{1}\end{bmatrix}\mathbf{w}(t), \tag{6}$$

with state transition matrix and process noise covariance over an interval Δt given by

$$\boldsymbol{\Phi}(\Delta t) = \begin{bmatrix}\mathbf{1} & \Delta t\,\mathbf{1}\\ \mathbf{0} & \mathbf{1}\end{bmatrix}, \qquad \mathbf{Q}(\Delta t) = \begin{bmatrix}\frac{1}{3}\Delta t^3\,\mathbf{Q}_c & \frac{1}{2}\Delta t^2\,\mathbf{Q}_c\\ \frac{1}{2}\Delta t^2\,\mathbf{Q}_c & \Delta t\,\mathbf{Q}_c\end{bmatrix}. \tag{7}$$

The local state at the two ends of the interval can be expressed in terms of the state variables. At t_i, the local pose variable is zero by definition, and its derivative equals the camera-frame velocity. At t_{i+1}, the derivative of the local variable is related to the camera-frame velocity through the inverse of the right Jacobian of SE(3), $\mathcal{J}_r^{-1}(\cdot)$ [29]. Hence,

$$\boldsymbol{\gamma}_i(t_i) = \begin{bmatrix}\mathbf{0}\\ \boldsymbol{\varpi}_i\end{bmatrix}, \qquad \boldsymbol{\gamma}_i(t_{i+1}) = \begin{bmatrix}\boldsymbol{\xi}_{i,i+1}\\ \mathcal{J}_r^{-1}(\boldsymbol{\xi}_{i,i+1})\,\boldsymbol{\varpi}_{i+1}\end{bmatrix}, \qquad \boldsymbol{\xi}_{i,i+1} = \text{Log}\left(\mathbf{T}_i^{-1}\mathbf{T}_{i+1}\right). \tag{8}$$

1. Motion Prior Factor

Consecutive states $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$ are connected by a binary GP prior factor with error

$$\mathbf{e}_{p,i} = \boldsymbol{\gamma}_i(t_{i+1}) - \boldsymbol{\Phi}(\Delta t)\,\boldsymbol{\gamma}_i(t_i) = \begin{bmatrix} \boldsymbol{\xi}_{i,i+1} - \Delta t\,\boldsymbol{\varpi}_i \\ \mathcal{J}_r^{-1}(\boldsymbol{\xi}_{i,i+1})\,\boldsymbol{\varpi}_{i+1} - \boldsymbol{\varpi}_i \end{bmatrix}, \tag{9}$$

and cost $\frac{1}{2}\,\mathbf{e}_{p,i}^T\,\mathbf{Q}(\Delta t)^{-1}\,\mathbf{e}_{p,i}$. Equation (9) penalizes deviations from constant camera-frame velocity, weighted by the expected accelerations encoded in $\mathbf{Q}_c$. Because the states are uniformly spaced, the same covariance $\mathbf{Q}(\Delta t)$ applies to every motion prior factor, except for the final interval, which may be shorter.

2. GP Interpolation

A key property of the GP prior is that the trajectory at any time $\tau \in [t_i, t_{i+1}]$ can be computed from the two neighboring states alone [10, 11]. The local state at τ is given by

$$\boldsymbol{\gamma}_i(\tau) = \boldsymbol{\Lambda}(\tau)\,\boldsymbol{\gamma}_i(t_i) + \boldsymbol{\Psi}(\tau)\,\boldsymbol{\gamma}_i(t_{i+1}), \tag{10}$$

with interpolation matrices

$$\boldsymbol{\Psi}(\tau) = \mathbf{Q}(\tau - t_i)\,\boldsymbol{\Phi}(t_{i+1} - \tau)^T\,\mathbf{Q}(t_{i+1} - t_i)^{-1}, \qquad \boldsymbol{\Lambda}(\tau) = \boldsymbol{\Phi}(\tau - t_i) - \boldsymbol{\Psi}(\tau)\,\boldsymbol{\Phi}(t_{i+1} - t_i). \tag{11}$$

The interpolated pose is then recovered from the first block of $\boldsymbol{\gamma}_i(\tau)$ as

$$\mathbf{T}(\tau) = \mathbf{T}_i\,\mathrm{Exp}\left(\boldsymbol{\xi}_i(\tau)\right), \tag{12}$$

where $\mathrm{Exp}(\cdot)$ is the exponential map from $\mathbb{R}^6$ to SE(3). The interpolated pose is therefore a function of the four variables $\mathbf{T}_i$, $\boldsymbol{\varpi}_i$, $\mathbf{T}_{i+1}$, and $\boldsymbol{\varpi}_{i+1}$, and it can be queried at any time without adding new variables to the problem.

E. Gauge and Scale Constraints

A monocular visual estimation problem is invariant to a global rigid-body transformation and to a global change of scale, which together form a seven-degree-of-freedom gauge freedom. To make the problem well posed, unary prior factors are placed on the first two state poses, $\mathbf{T}_0$ and $\mathbf{T}_1$, centered at their initial values from the front end. The prior on $\mathbf{T}_0$ has a standard deviation of 10^{-6} on all six components and fixes the global position and orientation. The prior on $\mathbf{T}_1$ constrains only its translation, with a standard deviation of 10^{-4}, and leaves its rotation effectively free, with a standard deviation of 10^3. This fixes the distance between the first two states, and hence the scale of the trajectory, to that of the front-end initialization.

F. Landmarks and Reprojection Factors

1. Landmarks and Observation Selection

The 3D landmarks reconstructed by ORB-SLAM3 are included as variables $\mathbf{p}_\ell \in \mathbb{R}^3$, expressed in the world frame and initialized at their front-end positions. The landmark observations used in the optimization are collected from two sources: all observations in keyframes, and observations in non-keyframe frames, of which every fourth frame is retained to limit the problem size. Because the states are no longer tied to keyframes, observations from both sources are associated with the image frame in which they were made and are treated identically. Observations whose reprojection error under the initial front-end poses exceeds 5 pixels are removed as outliers, and only landmarks with at least three remaining observations are retained. The image observations are the undistorted keypoint locations provided by the front end, so a pinhole projection model $\pi(\cdot)$ with known intrinsic calibration and no distortion is used.

2. Direct Reprojection Factors

When an observation $\mathbf{z}_{j\ell} \in \mathbb{R}^2$ of landmark ℓ is made in a frame j whose timestamp coincides with a state time t_i, it is connected directly to the state pose $\mathbf{T}_i$ and the landmark by a standard reprojection factor with error

$$\mathbf{e}_{j\ell} = \mathbf{z}_{j\ell} - \pi\left(\mathbf{T}_i^{-1}\,\mathbf{p}_\ell\right), \tag{13}$$

where the inverse transformation is applied to $\mathbf{p}_\ell$ in homogeneous coordinates. The error is weighted by an isotropic measurement noise covariance $\mathbf{R} = \sigma_z^2\,\mathbf{1}$ with $\sigma_z = 1$ pixel, and a Huber robust kernel [**?**] is applied to reduce the influence of remaining outliers.

3. Interpolated Reprojection Factors

Most observations are made in frames that lie strictly between two state times. Rather than introducing a new pose variable for each of these frames, the pose at the frame timestamp is obtained by GP interpolation. For an observation $\mathbf{z}_{j\ell}$ in frame j with timestamp $\tau_j \in (t_i, t_{i+1})$, the error is

$$\mathbf{e}_{j\ell} = \mathbf{z}_{j\ell} - \pi\left(\mathbf{T}(\tau_j)^{-1}\,\mathbf{p}_\ell\right), \tag{14}$$

where $\mathbf{T}(\tau_j)$ is the interpolated pose from Eqs. (10) and (12). Each interpolated reprojection factor is therefore connected to five variables: the two neighboring state poses $\mathbf{T}_i$ and $\mathbf{T}_{i+1}$, their velocities $\boldsymbol{\varpi}_i$ and $\boldsymbol{\varpi}_{i+1}$, and the landmark $\mathbf{p}_\ell$. These factors use the same noise covariance $\mathbf{R}$ with a Gaussian noise model, relying on the initial outlier rejection described above. They allow the image stream between states to inform the trajectory estimate, while the number of state variables remains proportional to the duration of the trajectory divided by Δt, rather than to the number of images.

G. Optimization and Trajectory Recovery

Collecting all factors, the maximum a posteriori estimate of the states and landmarks is obtained by solving the nonlinear least-squares problem

$$\min_{\{\mathbf{x}_i\},\,\{\mathbf{p}_\ell\}} \sum_{i=0}^{N-1} \left\|\mathbf{e}_{p,i}\right\|^2_{\mathbf{Q}(t_{i+1}-t_i)} + \sum_{(j,\ell)\in\mathcal{D}} \rho\left(\left\|\mathbf{e}_{j\ell}\right\|^2_{\mathbf{R}}\right) + \sum_{(j,\ell)\in\mathcal{I}} \left\|\mathbf{e}_{j\ell}\right\|^2_{\mathbf{R}} + \sum_{m=0}^{1} \left\|\mathbf{e}_{0,m}\right\|^2_{\mathbf{P}_{0,m}}, \tag{15}$$

where $\|\mathbf{e}\|^2_{\boldsymbol{\Sigma}} = \mathbf{e}^T\boldsymbol{\Sigma}^{-1}\mathbf{e}$, $\rho(\cdot)$ is the Huber cost function, $\mathcal{D}$ and $\mathcal{I}$ are the sets of direct and interpolated observations, respectively, and $\mathbf{e}_{0,m}$ and $\mathbf{P}_{0,m}$ are the errors and covariances of the gauge and scale priors on $\mathbf{T}_0$ and $\mathbf{T}_1$. The process noise power spectral density is set to $\mathbf{Q}_c = 0.1\,\mathbf{1}$. The problem is solved using the Levenberg–Marquardt algorithm in GTSAM, with a maximum of 100 iterations and relative and absolute error tolerances of 10^{-6}. Because each motion prior factor connects only consecutive states and each interpolated factor connects only the two neighboring states, the problem retains the sparse structure of standard bundle adjustment.

After optimization, the continuous-time trajectory is recovered by evaluating Eqs. (10) and (12) at the timestamp of every image frame, which yields a full-rate pose estimate from a much smaller number of state variables. The same procedure can be used to query the pose and velocity at any other time, such as the timestamps of the ground-truth measurements or of other sensors.

IV. Preliminary Results

This section presents preliminary results on three sequences of the EuRoC dataset. Section IV.A describes the experimental setup and evaluation protocol, Section IV.B compares the accuracy of the proposed framework with ORB-SLAM3 and VGGT-Ω, Section IV.C analyzes the effect of state placement, and Section IV.D discusses the computational cost.

A. Experimental Setup

The methods are evaluated on the MH01, MH03, and MH04 sequences of the EuRoC micro aerial vehicle dataset [14], which are classified by the dataset authors as easy, medium, and difficult, respectively. These sequences were recorded in an industrial machine hall and have durations of 184 s, 132 s, and 100 s. Only the images from the left camera (cam0) are used. Four trajectory estimates are compared on each sequence:

1) ORB-SLAM3 [13] in monocular mode, which provides the per-frame tracking poses used as the discrete-time baseline and as the initialization of both GP variants.
2) The keyframe-state GP variant, in which the GP states are placed at the ORB-SLAM3 keyframe times.
3) The proposed uniform-grid GP framework, in which the GP states are placed on a uniform temporal grid with spacing $\Delta t = 0.5$ s.

4) VGGT-Ω [7], a learning-based feed-forward model that predicts camera poses directly from images.
The two GP variants use the same landmarks, observations, outlier rejection, motion prior hyperparameters, and gauge constraints, so they differ only in the placement of the states. After optimization, both GP trajectories are queried at the timestamp of every image frame.

VGGT-Ω is applied to exactly the same set of images that provide observations to the uniform-grid GP framework. The images are undistorted with the EuRoC cam0 calibration and processed at a resolution of 512 pixels. Because the full image sequence cannot be processed in a single forward pass, the images are processed in overlapping chunks of 60 frames with an overlap of 20 frames, and consecutive chunks are joined with a similarity transformation estimated from the overlapping frames only. The rotation of this transformation is computed from the predicted camera orientations, and its scale is computed from the ratio of the predicted scene depths, so no ground-truth information is used to construct the VGGT-Ω trajectory.

All methods are evaluated with a common protocol. The evaluation is restricted to the frames for which all four methods provide a pose and ground truth is available within 5 ms, which results in 1339, 996, and 843 frames on MH01, MH03, and MH04, respectively. The ground-truth body poses are transformed to the cam0 frame using the camera-to-body extrinsic calibration of the dataset. Because the scale of a monocular trajectory is unobservable, each estimate is aligned to the ground truth with a single similarity transformation, Sim(3), over the entire sequence, computed with the method of Umeyama [**?**]. Four metrics are reported: the root mean square (RMS) and maximum of the absolute trajectory error (ATE) in translation; the RMS translational relative pose error (RPE) over a time interval of 1 s, which measures local accuracy independently of accumulated drift; and the RMS rotation error.

B. Comparison of Trajectory Accuracy

Table 2 reports the ATE of the four methods, and Fig. 1 summarizes the ATE RMS values. The proposed uniform-grid GP framework achieves the lowest ATE RMS and the lowest maximum error on all three sequences. Relative to the ORB-SLAM3 trajectory from which it was initialized, it reduces the ATE RMS by 28.1%, 31.3%, and 24.5% on MH01, MH03, and MH04, respectively, and on the difficult MH04 sequence it reduces the maximum error from 0.350 m to 0.232 m. VGGT-Ω produces ATE RMS values between 0.72 m and 1.06 m, which are 50, 44, and 22 times larger than those of the proposed framework on MH01, MH03, and MH04, respectively.

Table 2 Absolute trajectory error on the common frames after Sim(3) alignment

	MH01 (easy)		MH03 (medium)		MH04 (difficult)	
Method	RMS, m	Max, m	RMS, m	Max, m	RMS, m	Max, m
ORB-SLAM3	0.020	0.063	0.032	0.109	0.063	0.350
GP, keyframe states	0.046	0.449	0.220	1.666	0.070	0.469
GP, uniform grid (proposed)	**0.014**	**0.047**	**0.022**	**0.099**	**0.047**	**0.232**
VGGT-Ω	0.718	2.035	0.970	2.133	1.057	2.659

Table 3 reports the local translational accuracy and the rotation accuracy. The proposed framework has the lowest RPE on all three sequences, and its rotation error is within 0.02 deg of that of ORB-SLAM3. Since the front end already estimates the camera orientation accurately, the improvement provided by the GP framework is concentrated in the translation, which is the component most affected by the smoothness of the motion prior and by the additional observations from non-keyframe frames.

Table 3 Relative pose error over 1 s and rotation error on the common frames

	RPE (1 s) RMS, m			Rotation error RMS, deg		
Method	MH01	MH03	MH04	MH01	MH03	MH04
ORB-SLAM3	0.010	0.025	0.045	**1.20**	**1.35**	1.15
GP, keyframe states	0.062	0.133	0.074	2.36	2.24	2.17
GP, uniform grid (proposed)	**0.010**	**0.021**	**0.038**	1.22	1.36	**1.15**
VGGT-Ω	0.089	0.275	0.149	4.83	2.87	5.53

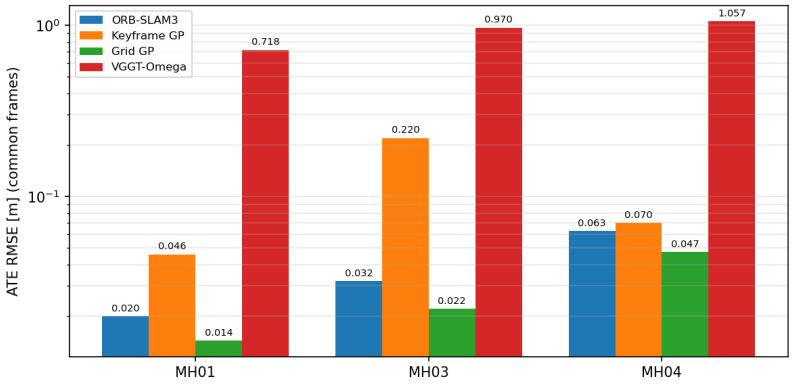


Fig. 1 Absolute trajectory error RMS of the four methods on the common frames of each sequence (logarithmic scale).

Figures 2–4 show the aligned trajectories and the error over time for ORB-SLAM3 and the two GP variants, and Figs. 5–7 show the corresponding results for VGGT-Ω, which are plotted separately because its errors are more than an order of magnitude larger. The ORB-SLAM3 and uniform-grid GP trajectories are visually indistinguishable from the ground truth at the scale of Figs. 2–4, whereas the VGGT-Ω trajectory in Figs. 5–7 deviates visibly in both the horizontal and the vertical directions. The error plots in Figs. 5–7 show that the error of VGGT-Ω is dominated by a slowly varying component that reaches 0.5 m to 2.7 m over tens of seconds, rather than by high-frequency noise. This interpretation is supported by the RPE: although the ATE of VGGT-Ω is 22 to 50 times larger than that of the proposed framework, its RPE over 1 s is only 4 to 13 times larger. VGGT-Ω therefore estimates the local motion within each chunk reasonably well, but it lacks the global consistency required over a complete flight. Each chunk has its own scale and reference frame, and the small errors in the similarity transformations that connect consecutive chunks accumulate along the trajectory. In contrast, the GP framework optimizes all states jointly against a single, globally consistent set of landmarks, so the scale and orientation are shared across the entire trajectory.

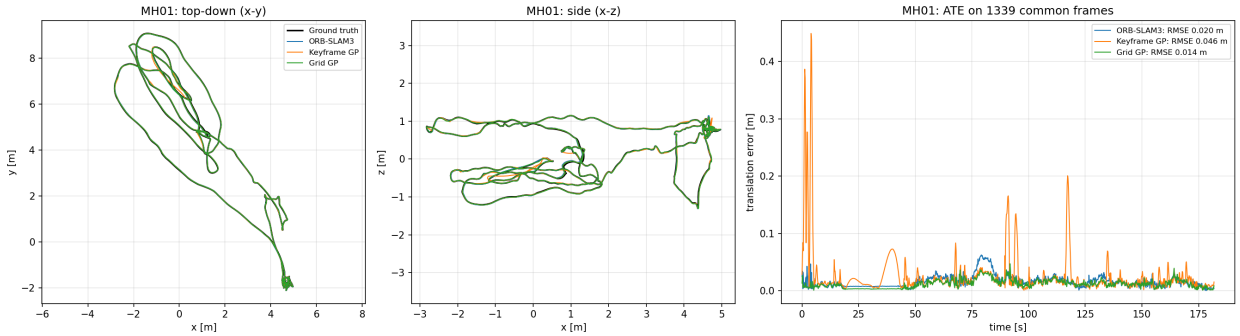


Fig. 2 MH01, ORB-SLAM3 and GP variants: aligned top-down (left) and side (center) trajectories, and translation error over time (right).

The VGGT-Ω results depend on the processing settings. In a preliminary test on MH01 with every fifth image as input, VGGT-Ω achieved an ATE RMS of 0.367 m, which is lower than the 0.718 m reported in Table 2 but still approximately 25 times larger than that of the proposed framework. The conclusion of the comparison therefore does not depend on the particular VGGT-Ω configuration. It should also be noted that VGGT-Ω is used here as an off-the-shelf model, without loop closure or global refinement, whereas the GP framework benefits from the globally consistent map produced by the ORB-SLAM3 front end. The comparison thus reflects the performance that each approach delivers in a complete trajectory estimation pipeline, rather than the accuracy of the learned model on short image sets.

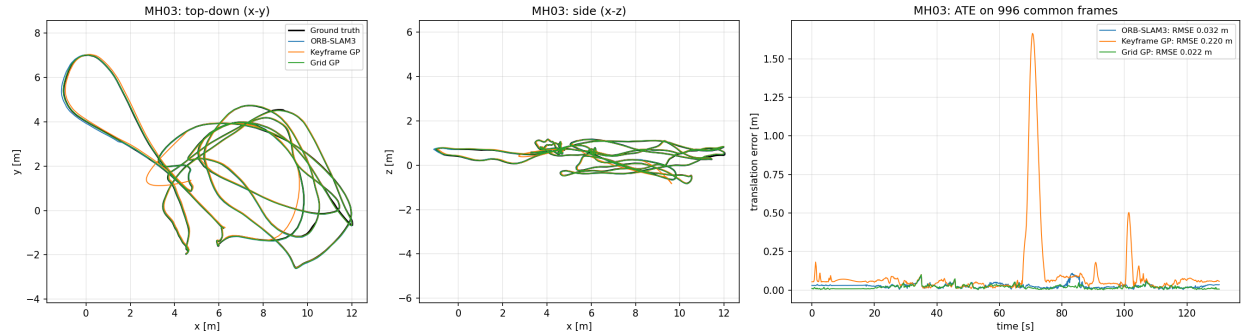


Fig. 3 MH03, ORB-SLAM3 and GP variants: aligned top-down (left) and side (center) trajectories, and translation error over time (right).

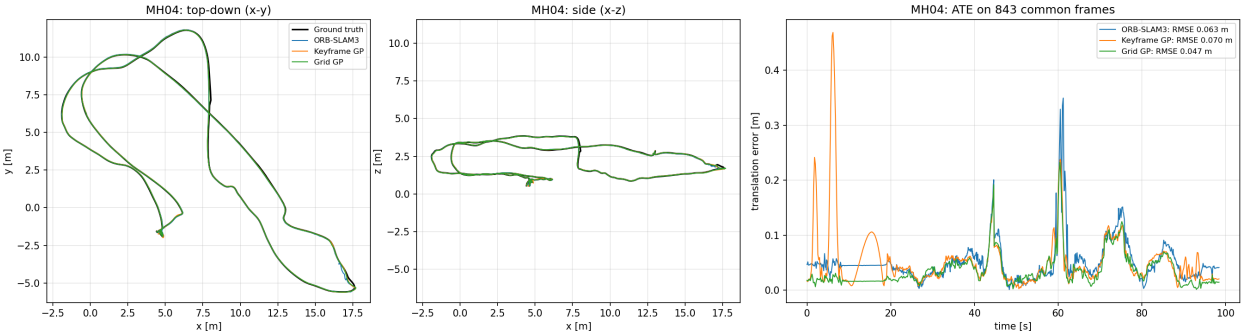


Fig. 4 MH04, ORB-SLAM3 and GP variants: aligned top-down (left) and side (center) trajectories, and translation error over time (right).

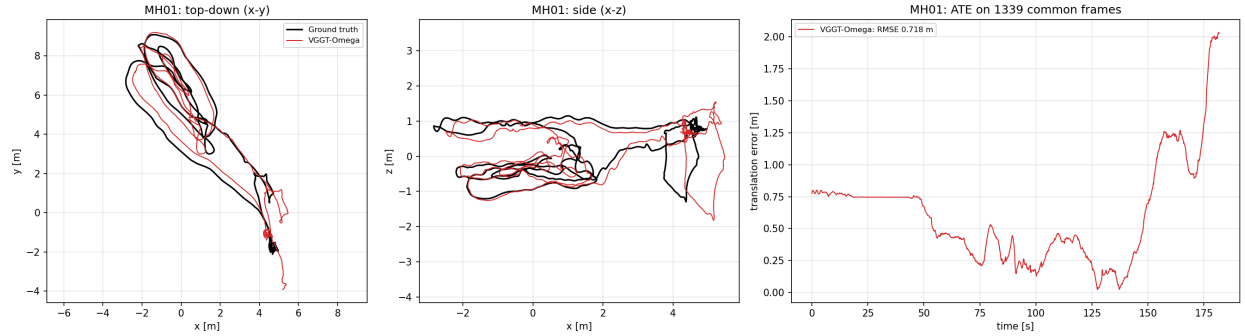


Fig. 5 MH01, VGGT-Ω: aligned top-down (left) and side (center) trajectories, and translation error over time (right).

C. Effect of State Placement

The keyframe-state GP variant has a higher ATE than ORB-SLAM3 on all three sequences, with an RMS error of 0.220 m and a maximum error of 1.67 m on MH03. Its rotation error is also approximately twice that of ORB-SLAM3. These errors arise from the interpolation between keyframes rather than from the keyframe estimates themselves: when the evaluation is restricted to the keyframe times, the keyframe-state variant reduces the ATE of the ORB-SLAM3 keyframes by 35.8%, 34.3%, and 13.0% on MH01, MH03, and MH04, respectively.

The cause is the irregular spacing of the keyframes selected by the front end. Although the mean interval between keyframes is between 0.30 s and 0.49 s on the three sequences, which is comparable to the 0.5 s grid spacing, the longest interval is 12.6 s on MH01, 9.5 s on MH03, and 9.6 s on MH04. These long intervals occur when ORB-SLAM3 inserts

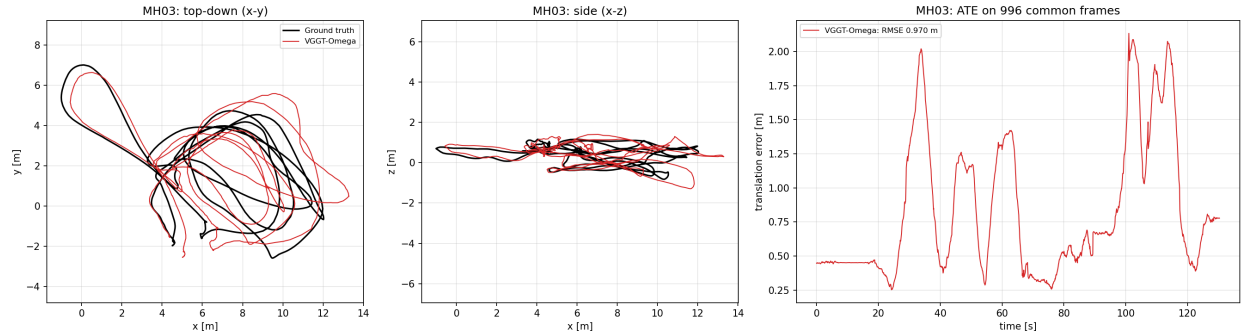


Fig. 6 MH03, VGGT-Ω: aligned top-down (left) and side (center) trajectories, and translation error over time (right).

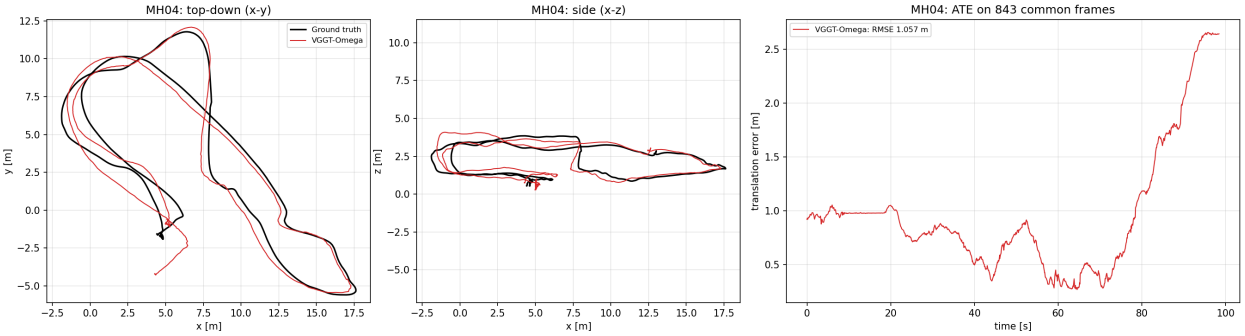


Fig. 7 MH04, VGGT-Ω: aligned top-down (left) and side (center) trajectories, and translation error over time (right).

no new keyframes because tracking remains reliable without them, which is typically the case when the camera moves slowly or revisits an already mapped area. Over such an interval, the keyframe-state variant represents the trajectory with only two states, and the WNOA prior cannot represent the actual motion of the vehicle, which includes several changes of direction over several seconds. The resulting interpolation errors appear as isolated spikes in the error plots of Figs. 2–4, such as the error of 1.67 m near 70 s on MH03.

The uniform grid avoids this failure mode because it bounds the interval over which the motion prior must bridge the trajectory to $\Delta t = 0.5$ s everywhere, while the interpolated reprojection factors from the intermediate frames constrain the motion within each interval. This improvement is not obtained by adding states: the uniform-grid variant uses 369, 266, and 201 states on MH01, MH03, and MH04, which is fewer than the 374, 309, and 341 keyframe states of the other variant.

D. Computational Cost

VGGT-Ω processed the images at an average rate of 2.7 images per second, with a peak graphics processing unit (GPU) memory usage of 9.5 GB on an NVIDIA RTX 4060 Ti. This memory requirement exceeds the capacity of most embedded processors used on small UAVs, and the processing rate is well below the 20 Hz camera rate. The GP framework, by comparison, runs on the central processing unit (CPU) without a GPU and requires memory only for a sparse optimization problem whose size is dominated by the landmarks and reprojection factors.

The current GP implementation is a batch optimization over the entire sequence, written in Python with the GTSAM bindings, and its processing time is therefore not representative of an onboard implementation. Because the number of states grows linearly with the duration of the trajectory and each factor involves at most two neighboring states, the problem is well suited to a sliding-window formulation with bounded computational cost, which will be developed and evaluated on embedded hardware in the final paper.

E. Summary of Preliminary Findings

The preliminary results support the hypothesis stated in Section I. On sequences of easy, medium, and difficult levels, the proposed uniform-grid GP framework achieves the lowest translation error of all methods considered: it reduces the ATE of its ORB-SLAM3 initialization by 24% to 31%, matches its rotation accuracy, and achieves an ATE 22 to 50 times lower than that of VGGT-Ω, while running without a GPU. The results also show that the placement of the GP states is critical, since placing the states at the front-end keyframes produces large interpolation errors over the long keyframe intervals that occur in practice. The final paper will extend the evaluation to all EuRoC sequences, implement and time a sliding-window version of the framework on embedded hardware, and analyze the sensitivity of the results to the grid spacing Δt and the power spectral density $\mathbf{Q}_c$.

V. Conclusion

This paper presented a continuous-time trajectory estimation framework for monocular quadcopter navigation, in which the trajectory is modeled with a white-noise-on-acceleration GP prior and estimated jointly with the landmarks in a sparse factor graph. Two design choices were found to be essential. First, placing the GP states on a uniform temporal grid, rather than at the keyframes selected by the visual front end, bounds the interval that the motion prior must bridge and avoids the large interpolation errors that occur over long keyframe gaps. Second, interpolated reprojection factors allow observations from all image frames to constrain the trajectory without adding a state for each frame.

The preliminary results on three EuRoC sequences show that the proposed framework improves the translation accuracy of its ORB-SLAM3 initialization by 24% to 31%, while preserving its rotation accuracy, and that it achieves an absolute trajectory error more than an order of magnitude lower than that of VGGT-Ω, without requiring a GPU. These results indicate that, for UAV navigation, an explicit and physically interpretable motion prior combined with a globally consistent map remains more accurate than current general-purpose learned geometry models, whose chunk-wise predictions accumulate drift over a complete flight.

Future work will focus on three directions: a sliding-window formulation of the framework that runs in real time on embedded hardware; an evaluation on all EuRoC sequences, together with an analysis of the sensitivity to the grid spacing and the power spectral density of the motion prior; and higher-order motion priors, such as the white-noise-on-jerk model, as well as the fusion of inertial measurements. The complementary strengths observed in this study also motivate hybrid approaches, in which learned models supply visual measurements to a continuous-time back end.

References

[1] Scaramuzza, D., and Fraundorfer, F., "Visual Odometry [Tutorial]," *IEEE Robotics & Automation Magazine*, 2011.

[2] Mur-Artal, R., Montiel, J. M. M., and Tardós, J. D., "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," *IEEE Transactions on Robotics*, 2015.

[3] Forster, C., Pizzoli, M., and Scaramuzza, D., "SVO: Fast Semi-Direct Monocular Visual Odometry," *IEEE International Conference on Robotics and Automation (ICRA)*, 2014.

[4] Qin, T., Li, P., and Shen, S., "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," *IEEE Transactions on Robotics*, 2018.

[5] Wang, S., Leroy, V., Cabon, Y., Chidlovskii, B., and Revaud, J., "DUSt3R: Geometric 3D Vision Made Easy," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[6] Wang, J., Chen, M., Karaev, N., Vedaldi, A., Rupprecht, C., and Novotny, D., "VGGT: Visual Geometry Grounded Transformer," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[7] Wang, J., Chen, M., Zhang, S., Karaev, N., Schönberger, J., Labatut, P., Bojanowski, P., Novotny, D., Vedaldi, A., and Rupprecht, C., "VGGT-Ω," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[8] Furgale, P., Barfoot, T. D., and Sibley, G., "Continuous-Time Batch Estimation Using Temporal Basis Functions," *IEEE International Conference on Robotics and Automation (ICRA)*, 2012.

[9] Tong, C. H., Furgale, P., and Barfoot, T. D., "Gaussian Process Gauss-Newton for Non-Parametric Simultaneous Localization and Mapping," *The International Journal of Robotics Research*, 2013.

[10] Barfoot, T. D., Tong, C. H., and Särkkä, S., "Batch Continuous-Time Trajectory Estimation as Exactly Sparse Gaussian Process Regression," *Robotics: Science and Systems (RSS)*, 2014.

[11] Anderson, S., and Barfoot, T. D., "Full STEAM Ahead: Exactly Sparse Gaussian Process Regression for Batch Continuous-Time Trajectory Estimation on SE(3)," *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015.

[12] Tang, T. Y., Yoon, D. J., and Barfoot, T. D., "A White-Noise-on-Jerk Motion Prior for Continuous-Time Trajectory Estimation on SE(3)," *IEEE Robotics and Automation Letters*, 2019.

[13] Campos, C., Elvira, R., Gómez Rodríguez, J. J., Montiel, J. M. M., and Tardós, J. D., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM," *IEEE Transactions on Robotics*, 2021.

[14] Burri, M., Nikolic, J., Gohl, P., Schneider, T., Rehder, J., Omari, S., Achtelik, M. W., and Siegwart, R., "The EuRoC Micro Aerial Vehicle Datasets," *The International Journal of Robotics Research*, 2016.

[15] Engel, J., Koltun, V., and Cremers, D., "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.

[16] Lovegrove, S., Patron-Perez, A., and Sibley, G., "Spline Fusion: A Continuous-Time Representation for Visual-Inertial Fusion with Application to Rolling Shutter Cameras," *British Machine Vision Conference (BMVC)*, 2013.

[17] Sommer, C., Usenko, V., Schubert, D., Demmel, N., and Cremers, D., "Efficient Derivative Computation for Cumulative B-Splines on Lie Groups," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[18] Mueggler, E., Gallego, G., Rebecq, H., and Scaramuzza, D., "Continuous-Time Visual-Inertial Odometry for Event Cameras," *IEEE Transactions on Robotics*, 2018.

[19] Dong, J., Boots, B., and Dellaert, F., "Sparse Gaussian Processes for Continuous-Time Trajectory Estimation on Matrix Lie Groups," *arXiv preprint arXiv:1705.06020*, 2017.

[20] Teed, Z., and Deng, J., "DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras," *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[21] Leroy, V., Cabon, Y., and Revaud, J., "Grounding Image Matching in 3D with MASt3R," *European Conference on Computer Vision (ECCV)*, 2024.

[22] Zhang, J., Herrmann, C., Hur, J., Jampani, V., Darrell, T., Cole, F., Sun, D., and Yang, M.-H., "MonST3R: A Simple Approach for Estimating Geometry in the Presence of Motion," *International Conference on Learning Representations (ICLR)*, 2025.

[23] Li, Z., Tucker, R., Cole, F., Wang, Q., Jin, L., Ye, V., Kanazawa, A., Holynski, A., and Snavely, N., "MegaSaM: Accurate, Fast, and Robust Structure and Motion from Casual Dynamic Videos," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[24] Xu, Z., Zhou, S., Luo, Y., and Loy, C. C., "HiP3R: Hierarchical Pose Graphs for Long-Horizon 3D Reconstruction," , 2026. URL `https://arxiv.org/abs/2610.31635`.

[25] Miao, X., Zhao, W., Lu, T., Xu, L., Yu, M., Long, Y., Pang, J., and Dong, J., "TrajVG: 3D Trajectory-Coupled Visual Geometry Learning," , 2026. URL `https://arxiv.org/abs/2602.04439`.

[26] Dellaert, F., and Kaess, M., "Factor Graphs for Robot Perception," *Foundations and Trends in Robotics*, 2017.

[27] Dellaert, F., "Factor Graphs and GTSAM: A Hands-on Introduction," Tech. Rep. GT-RIM-CP&R-2012-002, Georgia Institute of Technology, 2012.

[28] Rasmussen, C. E., and Williams, C. K. I., *Gaussian Processes for Machine Learning*, MIT Press, Cambridge, MA, 2006.

[29] Solà, J., Deray, J., and Atchuthan, D., "A Micro Lie Theory for State Estimation in Robotics," *arXiv preprint arXiv:1812.01537*, 2018.